

Klasifikacija_teksta

December 1, 2025

0.1 # Metode strojnog učenja u obradi teksta

```
[1]: # preambula
# paketi
import sklearn
import nltk
import pandas as pd
import re

# scikit-learn funkcije
from sklearn.feature_extraction.text import CountVectorizer # pretvara
    ↪ dokumente u vektore frekvencija tokena
from sklearn.feature_extraction.text import TfidfVectorizer #

# NLTK funkcije
from nltk.stem.porter import PorterStemmer # korijenovatelj engleskih riječi
from sklearn.cluster import KMeans # algoritam klasteriranja
from pprint import pprint

# opcije
pd.options.display.max_columns = 30
%matplotlib inline
```

0.2 Analiziranje teksta

Pristup analize: *rečenica kao neporedani skup riječi* (*“bag-of-words” analiza*). Riječi se vektoriziraju kao numeričke vrijednosti prema koracima: 1. **Raščlamba (tokenizacija)**: pretvara riječi/fraze u niz tokena i pridružuje numeričke identifikatore; `nltk.tokenize` 2. **Prebrojavanje**: prebrojavanje pojave tokena u dokumentima `scikit-learn` 3. **Normalizacija**: ponderiranje važnosti tokena u dokumentima `scikit-learn`

0.3 Pristup strojnog učenja

- frekvencija pojedinog tokena u dokumentu: **svojstvo** (engl. **feature**):
- vektor frekvencija tokena: **uzorak**
- koje **statistike** u postupku normalizacije reflektiraju koliko je važna neka riječ u dokumentu nekog korpusa? TF-IDF

CILJ: Odrediti sličnost dokumenata

Primjer vlastitog korpusa dokumenata:

```
[2]: # primjer teksta
from data import texts1 as texts
pprint(texts)

['Penny bought bright blue fishes.',
 'Penny bought bright blue and orange fish.',
 'The cat ate a fish at the store.',
 'Penny went to the store. Penny ate a bug. Penny saw a fish.',
 'It meowed once at the bug, it is still meowing at the bug and the fish',
 'The cat is at the fish store. The cat is orange. The cat is meowing at the '
 'fish.',
 'Penny is a fish']
```

Vektorizirajmo dokumente s obzirom na pojavnost tokena uz pomoć sklearn-ova `CountVector`

```
[4]: count_vectorizer = CountVectorizer()
# tokeniziraj i prebroj
X = count_vectorizer.fit_transform(texts)
# koristi pandas za prikaz
#pd.DataFrame(X.toarray())
# poboljšaj prikaz?
pd.DataFrame(X.toarray(), columns=count_vectorizer.get_feature_names_out())
```

```
[4]:
```

	and	at	ate	blue	bought	bright	bug	cat	fish	fishes	is	it	meowed	\
0	0	0	0	1	1	1	0	0	0	1	0	0	0	
1	1	0	0	1	1	1	0	0	1	0	0	0	0	
2	0	1	1	0	0	0	0	1	1	0	0	0	0	
3	0	0	1	0	0	0	1	0	1	0	0	0	0	
4	1	2	0	0	0	0	2	0	1	0	1	2	1	
5	0	2	0	0	0	0	0	3	2	0	3	0	0	
6	0	0	0	0	0	0	0	0	1	0	1	0	0	

	meowing	once	orange	penny	saw	still	store	the	to	went
0	0	0	0	1	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	0	0
2	0	0	0	0	0	0	1	2	0	0
3	0	0	0	3	1	0	1	1	1	1
4	1	1	0	0	0	1	0	3	0	0
5	1	0	1	0	0	0	1	5	0	0
6	0	0	0	1	0	0	0	0	0	0

Iz prethodnog vidimo da retci odgovaraju našim dokumentima, a stupci riječima unutar rečenice. Uočimo da u tekstu ima podosta **učestalih riječi** (engl. stopwords) koje želimo filtrirati!

```
[6]: # novi vektorizer
count_vectorizer = CountVectorizer(stop_words='english')
X = count_vectorizer.fit_transform(texts)
pd.DataFrame(X.toarray(), columns=count_vectorizer.get_feature_names_out())
```

	ate	blue	bought	bright	bug	cat	fish	fishes	meowed	meowing	orange	\
0	0	1	1	1	0	0	0	1	0	0	0	
1	0	1	1	1	0	0	1	0	0	0	1	
2	1	0	0	0	0	1	1	0	0	0	0	
3	1	0	0	0	1	0	1	0	0	0	0	
4	0	0	0	0	2	0	1	0	1	1	0	
5	0	0	0	0	0	3	2	0	0	1	1	
6	0	0	0	0	0	0	1	0	0	0	0	

	penny	saw	store	went
0	1	0	0	0
1	1	0	0	0
2	0	0	1	0
3	3	1	1	1
4	0	0	0	0
5	0	0	1	0
6	1	0	0	0

Dodatna filtracija tokena: > normalizacija (lematizacija ili korijenovanje) tokena.

```
[18]: from nltk.stem import WordNetLemmatizer

# lematizator
lemmatizer = WordNetLemmatizer()
# tokenizator
def tokenizer(text):
    words = re.sub(r'[^A-Za-z0-9\-\_]', " ", text).lower().split() # prave riječi u
    ↪ tekstu
    words = [lemmatizer.lemmatize(word) for word in words]
    return words
# vektorizator
count_vectorizer = CountVectorizer(stop_words='english', tokenizer=tokenizer)
X = count_vectorizer.fit_transform(texts)
pd.DataFrame(X.toarray(), columns=count_vectorizer.get_feature_names_out())
```

```
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:528: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
    warnings.warn(
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:409: UserWarning: Your stop_words
may be inconsistent with your preprocessing. Tokenizing the stop words generated
tokens ['ha', 'le', 'u', 'wa'] not in stop_words.
```

```
warnings.warn(
```

```
[18]:
```

	ate	blue	bought	bowl	bright	bug	cat	closed	eat	fish	love	\
0	0	1	1	0	1	0	0	0	0	1	0	
1	0	1	1	1	1	0	0	0	0	0	0	
2	1	0	0	0	0	0	1	0	0	1	0	
3	1	0	0	0	0	1	0	0	0	1	0	
4	0	0	0	0	0	2	0	0	0	1	0	
5	0	0	0	0	0	0	3	0	0	2	0	
6	0	0	0	0	0	0	0	0	0	1	0	
7	0	0	0	0	0	0	1	0	0	1	1	
8	0	0	0	0	0	0	0	1	0	0	0	
9	0	0	0	0	0	0	0	0	0	0	0	
10	0	0	0	0	0	1	1	0	3	1	0	

	meowed	meowing	old	orange	penny	saw	store	tree	went
0	0	0	0	0	1	0	0	0	0
1	0	0	0	1	1	0	0	0	0
2	0	0	0	0	0	0	1	0	0
3	0	0	0	0	3	1	1	0	1
4	1	1	0	0	0	0	0	0	0
5	0	1	0	1	0	0	1	0	0
6	0	0	0	0	1	0	0	0	0
7	0	0	0	0	4	0	0	0	0
8	0	0	0	0	0	0	1	0	0
9	0	0	1	0	0	0	0	1	0
10	0	0	0	0	0	0	0	0	0

0.3.1 TF-IDF

Term-Frequency-Inverse-Document-Frequency je statistika koja se koristi za vrednovanje koliko neka riječ nosi informacije u korpusu s obzirom na njenu pojavnost:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

0.4 Term Frequency (Frekvencija pojma)

Utvrdivanje relativne pojavnosti tokena t u dokumentu d :

$$\text{tf}(t, d) = \frac{f(t, d)}{\sum_{v \in d} f(v, d)}$$

gdje je $f(v, d)$ broj pojave tokena v u dokumentu d .

Koristimo sklearn-ov `TfidfVectorizer`.

```
[9]: tfidf_vectorizer = TfidfVectorizer(stop_words='english', tokenizer=tokenizer,
    ↪ use_idf=False, norm='l1')
X = tfidf_vectorizer.fit_transform(texts)
df=pd.DataFrame(X.toarray(), columns = tfidf_vectorizer.get_feature_names_out())
```

```
pprint(texts)
df
```

```
['Penny bought bright blue fishes.',
 'Penny bought bright blue and orange fish.',
 'The cat ate a fish at the store.',
 'Penny went to the store. Penny ate a bug. Penny saw a fish.',
 'It meowed once at the bug, it is still meowing at the bug and the fish',
 'The cat is at the fish store. The cat is orange. The cat is meowing at the '
 'fish.',
 'Penny is a fish']
```

```
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:528: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
  warnings.warn(
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:409: UserWarning: Your stop_words
may be inconsistent with your preprocessing. Tokenizing the stop words generated
tokens ['ha', 'le', 'u', 'wa'] not in stop_words.
  warnings.warn(
```

```
[9]:
```

	ate	blue	bought	bright	bug	cat	fish	meowed \
0	0.000000	0.200000	0.200000	0.200000	0.000000	0.000	0.200000	0.0
1	0.000000	0.166667	0.166667	0.166667	0.000000	0.000	0.166667	0.0
2	0.250000	0.000000	0.000000	0.000000	0.000000	0.250	0.250000	0.0
3	0.111111	0.000000	0.000000	0.000000	0.111111	0.000	0.111111	0.0
4	0.000000	0.000000	0.000000	0.000000	0.400000	0.000	0.200000	0.2
5	0.000000	0.000000	0.000000	0.000000	0.000000	0.375	0.250000	0.0
6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000	0.500000	0.0

	meowing	orange	penny	saw	store	went
0	0.000	0.000000	0.200000	0.000000	0.000000	0.000000
1	0.000	0.166667	0.166667	0.000000	0.000000	0.000000
2	0.000	0.000000	0.000000	0.000000	0.250000	0.000000
3	0.000	0.000000	0.333333	0.111111	0.111111	0.111111
4	0.200	0.000000	0.000000	0.000000	0.000000	0.000000
5	0.125	0.125000	0.000000	0.000000	0.125000	0.000000
6	0.000	0.000000	0.500000	0.000000	0.000000	0.000000

0.5 Inverse Document Frequency (Inverzna dokumenta frekvencija)

Mjera koliko informacije nosi token t s obzirom na pojavnost u svim dokumentima korpusa D :

$$\text{idf}(t) = -\log \frac{\text{df}(t, D)}{|D|}$$

gdje je $\text{df}(t, D)$ broj dokumenata u D koje sadrži token t .

Paket `sklearn` omogućava korištenje FT-IDF s opcijom `use_idf=True` u `TFidfVectorizer` metodi za vektorizaciju zajedno s odabirom normalizacije vektora korištenjem L_1 ili L_2 norme.

Napomena: `sklearn` implementira `TFidfVectorizer` `idf()` tako da u brojnik i nazivnik doda 1 kako bi se izbjeglo dijeljenje s 0.

```
[10]: # iskoristimo idf
idf_vectorizer = TfidfVectorizer(stop_words='english', tokenizer=tokenizer,
    ↪use_idf=True, norm='l2')
X = idf_vectorizer.fit_transform(texts)
df = pd.DataFrame(X.toarray(), columns=idf_vectorizer.get_feature_names_out())
df
```

```
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:528: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
  warnings.warn(
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:409: UserWarning: Your stop_words
may be inconsistent with your preprocessing. Tokenizing the stop words generated
tokens ['ha', 'le', 'u', 'wa'] not in stop_words.
  warnings.warn(
```

```
[10]:
```

	ate	blue	bought	bright	bug	cat	fish \
0	0.000000	0.512612	0.512612	0.512612	0.000000	0.000000	0.258786
1	0.000000	0.456170	0.456170	0.456170	0.000000	0.000000	0.230292
2	0.578752	0.000000	0.000000	0.000000	0.000000	0.578752	0.292176
3	0.303663	0.000000	0.000000	0.000000	0.303663	0.000000	0.153301
4	0.000000	0.000000	0.000000	0.000000	0.772313	0.000000	0.194947
5	0.000000	0.000000	0.000000	0.000000	0.000000	0.840166	0.282766
6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.562463

	meowed	meowing	orange	penny	saw	store	went
0	0.000000	0.000000	0.000000	0.380417	0.000000	0.000000	0.000000
1	0.000000	0.000000	0.456170	0.338530	0.000000	0.000000	0.000000
2	0.000000	0.000000	0.000000	0.000000	0.000000	0.494698	0.000000
3	0.000000	0.000000	0.000000	0.676058	0.365821	0.259561	0.365821
4	0.465201	0.386156	0.000000	0.000000	0.000000	0.000000	0.000000
5	0.000000	0.280055	0.280055	0.000000	0.000000	0.239382	0.000000
6	0.000000	0.000000	0.000000	0.826823	0.000000	0.000000	0.000000

1 Nenadzirano učenje

Princip klasteriranja: korištenjem *k*-Means algoritma ([docs](#)) cilj nam je odrediti *k* klastera dokumenata našeg korpusa tako da ukupna udaljenost između dokumenata istog klastera bude najmanja moguća!

```
[19]: # ponovimo podatke ...
vectorizer = TfidfVectorizer(use_idf=True, tokenizer=tokenizer,
    ↪stop_words='english')
X = vectorizer.fit_transform(texts)

/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:528: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
    warnings.warn(
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:409: UserWarning: Your stop_words
may be inconsistent with your preprocessing. Tokenizing the stop words generated
tokens ['ha', 'le', 'u', 'wa'] not in stop_words.
    warnings.warn(
```

```
[20]: # algoritam klasteriranja
from sklearn.cluster import KMeans
# broj klastera 2
number_of_clusters = 2
km = KMeans(n_clusters=number_of_clusters)
km.fit(X)

/opt/anaconda3/lib/python3.8/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
    warnings.warn(
```

```
[20]: KMeans(n_clusters=2)
```

```
[21]: # uljepšajmo prikaz podataka
results = pd.DataFrame()
results['text'] = texts
results['category'] = km.labels_ # klaster oznake
results
```

```
[21]:
```

	text	category
0	Penny bought bright blue fishes.	0
1	Penny bought bright blue and orange bowl.	0
2	The cat ate a fish at the store.	1
3	Penny went to the store. Penny ate a bug. Penn...	0
4	It meowed once at the bug, it is still meowing...	1
5	The cat is at the fish store. The cat is orang...	1
6	Penny is a fish.	0
7	Penny Penny she loves fishes Penny Penny is no...	0
8	The store is closed now.	1
9	How old is that tree?	0
10	I do not eat fish I do not eat cats I only eat...	1

```
[23]: # primjer 2
from data import texts2 as texts

pprint(texts)

# inicijalizirajmo vektor s najviše 2 istaknuta svojstva: 2 najčešće riječi u
↳ korpusu
vectorizer = TfidfVectorizer(use_idf=True, max_features=2, tokenizer=tokenizer,
↳ stop_words='english')
X = vectorizer.fit_transform(texts)

# klasteriraj
number_of_clusters = 3
km = KMeans(n_clusters=number_of_clusters)
km.fit(X)
# citaj podatke
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
df['category']=km.labels_
df
```

```
['Penny bought bright blue fishes.',
 'Penny bought bright blue and orange bowl.',
 'The cat ate a fish at the store.',
 'Penny went to the store. Penny ate a bug. Penny saw a fish.',
 'It meowed once at the bug, it is still meowing at the bug and the fish',
 'The cat is at the fish store. The cat is orange. The cat is meowing at the '
 'fish.',
 'Penny is a fish.',
 'Penny Penny she loves fishes Penny Penny is no cat.',
 'The store is closed now.',
 'How old is that tree?',
 'I do not eat fish I do not eat cats I only eat bugs']

/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:528: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
  warnings.warn(
/opt/anaconda3/lib/python3.8/site-
packages/sklearn/feature_extraction/text.py:409: UserWarning: Your stop_words
may be inconsistent with your preprocessing. Tokenizing the stop words generated
tokens ['ha', 'le', 'u', 'wa'] not in stop_words.
  warnings.warn(
/opt/anaconda3/lib/python3.8/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
  warnings.warn(
```



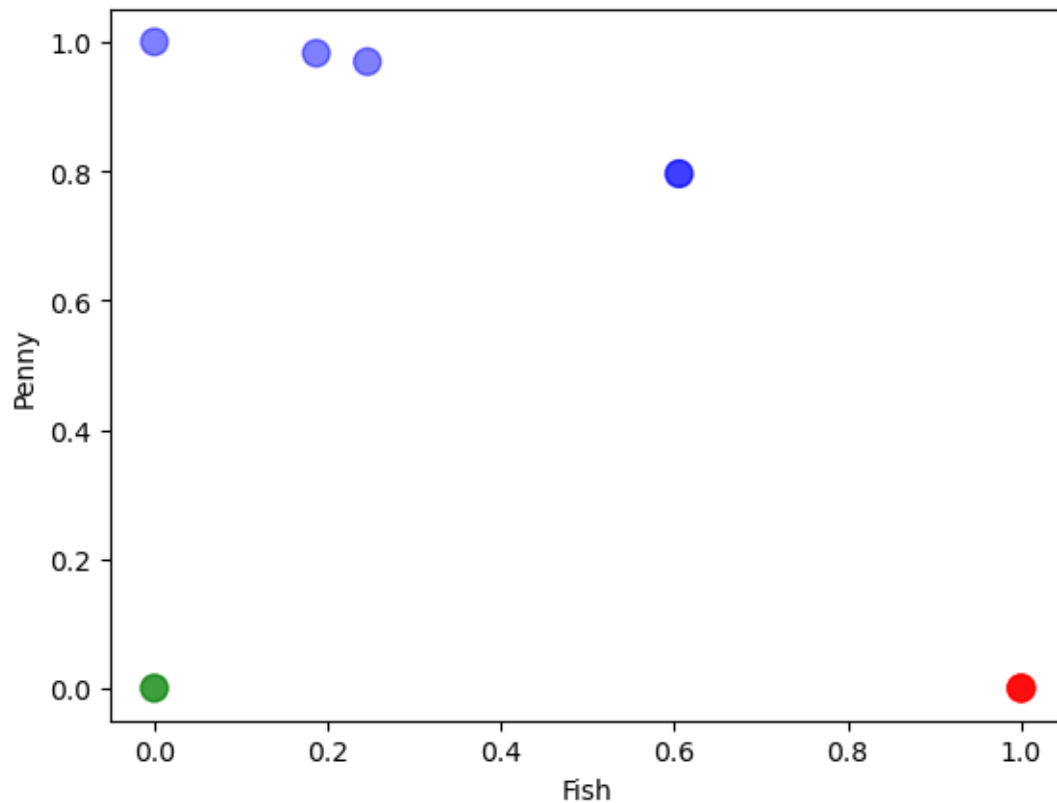
```
[23]:
```

	fish	penny	category
0	0.605349	0.795961	1
1	0.000000	1.000000	1
2	1.000000	0.000000	0
3	0.245735	0.969337	1
4	1.000000	0.000000	0
5	1.000000	0.000000	0
6	0.605349	0.795961	1
7	0.186785	0.982401	1
8	0.000000	0.000000	2
9	0.000000	0.000000	2
10	1.000000	0.000000	0

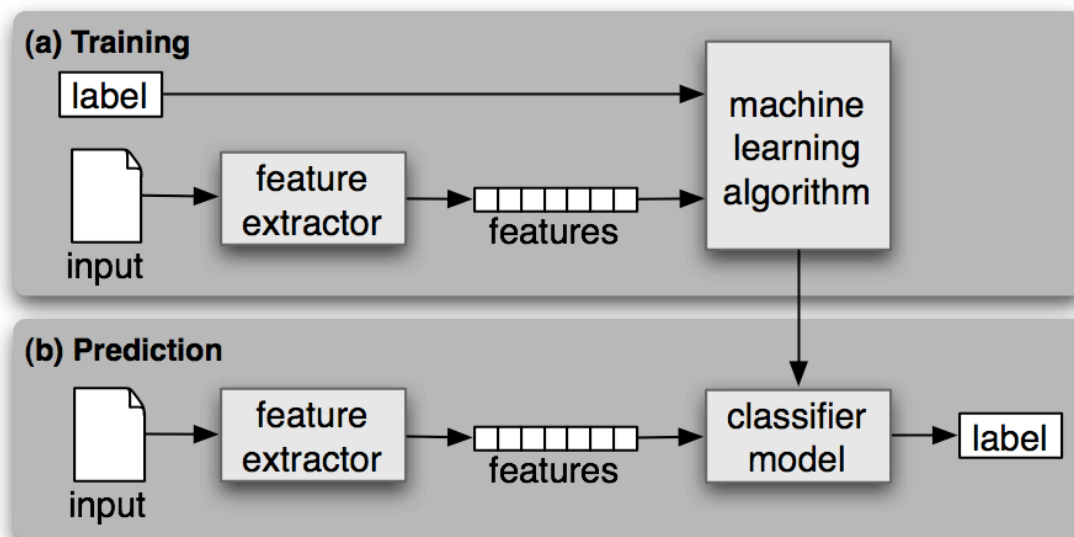
```
[24]: # ilustracija klastera
# boje klastera
color_list = ['r', 'b', 'g']
colors = [color_list[i] for i in df['category']]

ax = df.plot(kind='scatter', x='fish', y='penny', alpha=0.5, s=100, c=colors)
ax.set_xlabel("Fish")
ax.set_ylabel("Penny")
```

```
[24]: Text(0, 0.5, 'Penny')
```



2 Nadzirano učenje



- **Klasifikacija** = zadatak utvrđivanja grupe kojoj pripada dani podatak.
- Postupak može biti *nenadziran* ili *nadziran*: **non-supervised/supervised**

3 Primjer 1: Klasifikacija teksta

Klasteriranjem podataka dobili smo **tematske kategorije** dokumenata unutar korpusa. > **Pitanje**: kako odrediti kojoj kategoriji pripada novopridošli (nekategorizirani) dokument?

Odgovor: Primijeniti K-Nearest Neighbour klasifikator ([docs](#))!

```
[25]: from data import texts2 as texts

# Bag-of-Words
vectorizer = TfidfVectorizer(use_idf=True, stop_words='english')
X = vectorizer.fit_transform(texts)

# klasifikator
text = 'Penny likes to fish a fish.'

from sklearn.cluster import KMeans
# broj klastera
number_of_clusters = 4
km = KMeans(n_clusters=number_of_clusters)
km.fit(X)
```

```

# ispisi rezultat
results = pd.DataFrame()
results['text'] = texts
results['category'] = km.labels_
results

from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=3)
classifier.fit(X, km.labels_)
# test uzorak
test = vectorizer.transform([text])
#print(test.toarray())

# predikcija
category = classifier.predict(test)
print('Dokument je pridruzen klasteru: ', category)

```

Dokument je pridruzen klasteru: [2]

```

/opt/anaconda3/lib/python3.8/site-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in
1.4. Set the value of `n_init` explicitly to suppress the warning
warnings.warn(

```

3.0.1 Primjer 2 | identifikacija roda

Muška i ženska imena imaju nekih bitnih razlika: imena koja završavaju na a,e,i su većim dijelom ženska imena, dok imena koja završavaju na k,o,r,s su vjerojatnije muška...

3.0.2 Naivni Bayesov klasifikator

Za dani ulaz x sa svojstvima F Naivni Bayes pridružuje labelu $l \in L$ koja maksimizira vjerojatnost $P(l|F)$. Na temelju trening skupa računa

$$P(l|F) = P(F, l) / P(F)$$

gdje $P(F) = \sum_{l \in L} P(F, l)$. Za računanje $P(F, l)$ koristi prior distribuciju $P(l)$ (vjerojatnost oznake l) uz pretpostavku nezavisnosti svojstava:

$$P(F, l) = P(l) \times \prod_{f \in F} P(f|l)$$

Vjerojatnosti $P(f|l)$ i $P(l)$ računa na temelju frekvencija oznaka i svojstava u trening skupu. Npr. $P(f, l) = \text{count}(f, l) / \text{count}(l)$. U praksi, sofisticirane tehnike koriste se za zaglađivanje tih funkcija (izbjegavanje 0 vrijednosti).

Sljedeće funkcija određuje svojstvo zadnjeg slova u riječi.

```

[ ]: def gender_features(word):
      return {'last_letter': word[-1]}

```

```
gender_features('shrek')
```

Učitajmo sada skup podataka koji pored imena sadrži i oznake grupe.

```
[ ]: from nltk.corpus import names
import random

names = [(name, 'male') for name in names.words('male.txt')] + [(name, 'female') for name in names.words('female.txt')]
random.shuffle(names)
print(names[1:10])
```

Podijelimo `names` uz pomoć ekstraktora svojstva u 2 skupa svojstava: **skup za treniranje** i **skup za testiranje**. Iskoristimo dostupni klasifikator `NaiveBayesClassifier` (objasniti ćemo poslije...)

```
[ ]: # skup svojstava
featuresets = [(gender_features(n), g) for (n,g) in names]
# podjela u treniranje i test
train_set, test_set = featuresets[500:], featuresets[:500]
```

```
[ ]: # definiran model klasifikacije na temelju podataka
classifier = nltk.NaiveBayesClassifier.train(train_set)
```

Provjeriti klasifikator s novim pojmom. Kolika je preciznost klasifikatora?

```
[ ]: # provjeri klasifikaciju na novim pojmovima
classifier.classify(gender_features('Neo'))
classifier.classify(gender_features('Trinity'))
classifier.classify(gender_features('Cathleen'))

# provjeri preciznost na test skupu
nltk.classify.accuracy(classifier, test_set)
# koja slova najviše diskriminiraju imena?
classifier.show_most_informative_features(5)
```

Problem odabira **dobrog skupa svojstava**: *overfitting* vs *underfitting*

Analiziranje loše klasifikacije možemo probati odabrati dobar skup svojstava uz pomoć **validacijskog skupa** podataka

```
[ ]: # podaci
random.shuffle(names)
train_names = names[1500:]
devtest_names = names[500:1500] # na ovom skupu radimo analizu greške
test_names = names[:500]
```

```
[ ]: # feature extractor
def gender_features(word):
    return {'last_letter': word[-1]}
```

```

# model
train_set = [(gender_features(n), g) for (n,g) in train_names]
devtest_set = [(gender_features(n), g) for (n,g) in devtest_names]
test_set = [(gender_features(n), g) for (n,g) in test_names]
# treniranje
classifier = nltk.NaiveBayesClassifier.train(train_set)
# analiza greške
print(nltk.classify.accuracy(classifier, devtest_set))
#print(nltk.classify.accuracy(classifier, test_set))

# ispitati greške
errors = []
for (name, tag) in devtest_names:
    guess = classifier.classify(gender_features(name))
    if guess != tag:
        errors.append( (tag, guess, name) )

for (tag, guess, name) in sorted(errors):
    print('correct=%-8s guess=%-8s name=%-30s' % (tag, guess, name))

```

```
[ ]: classifier.classify(gender_features('Alys'))
```

Što nam govori ovaj primjer? > Neki sufiksi koji imaju barem 2 člana bolje karakteriziraju ime, npr. Cathle(en)-> female

```

[ ]: def gender_features2(word):
    return {'suffix1':word[-1:], 'suffix2':word[-2:]}

# model
train_set = [(gender_features2(n), g) for (n,g) in train_names]
devtest_set = [(gender_features2(n), g) for (n,g) in devtest_names]
test_set = [(gender_features2(n), g) for (n,g) in test_names]
# treniranje
classifier = nltk.NaiveBayesClassifier.train(train_set)
# analiza greške
print(nltk.classify.accuracy(classifier, devtest_set))
#print(nltk.classify.accuracy(classifier, test_set))

# ispitati greške
errors = []
for (name, tag) in devtest_names:
    guess = classifier.classify(gender_features2(name))
    if guess != tag:
        errors.append( (tag, guess, name) )

for (tag, guess, name) in sorted(errors):
    print('correct=%-8s guess=%-8s name=%-30s' % (tag, guess, name))

```

```
[ ]: classifier.classify(gender_features('Alys'))
```